

Stop writing the same API call

5 times!



@cosmiclearning1111



● You've probably done this:

```
useEffect(() => {  
  fetch('/api/data')  
    .then(res => res.json())  
    .then(setData);  
}, []);
```

[copy](#)

Copy-pasted across 5 components...

- ✗ No shared loading state
- ✗ No error handling
- ✗ No debounce
- ✗ Hard to test



@cosmiclearning1111



The better way

- ✓ Use a custom hook
- ✓ Debounce API calls
- ✓ Centralize loading + error states
- ✓ Cleaner component code
- ✓ Easier testing



@cosmiclearning1111



The Reusable Hook

```
// useFetchUsers.ts
import { useState, useEffect } from 'react';
import debounce from 'lodash.debounce';

export const useFetchUsers = (search: string) => {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const fetchData = debounce(async () => {
    setLoading(true);
    try {
      const res = await fetch(`/api/users?q=${search}`);
      const json = await res.json();
      setData(json);
    } catch (err) {
      setError(err);
    } finally {
      setLoading(false);
    }
  }, 300);

  useEffect(() => {
    fetchData();
    return fetchData.cancel;
  }, [search]);

  return { data, loading, error };
};
```

copy

The Usage in Component

```
const UserList = ({ search }) => {  
  const { data, loading, error } = useFetchUsers(search);  
  
  if (loading) return <Spinner />;  
  if (error) return <ErrorUI />;  
  
  return (  
    <ul>  
      {data.map(user => <li key={user.id}>{user.name}</li>)}  
    </ul>  
  );  
};
```

copy

Just plug it in, no boilerplate repeated.

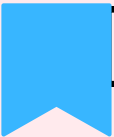
Bonus Tips

- Extract multiple hooks for different endpoints
- Wrap with Zustand/Context if it's app-wide
- Add caching with SWR or React Query
- Easier to unit test



@cosmiclearning1111





Save for later

Tired of boilerplate?
Start building like a scalable React dev.

Follow for more architecture tips!



@cosmiclearning1111

